**OBJECT ORIENTED PROGRAMMING**
**CCS20704, DCS20504, CCS23803**

| | |
|---|---|
| **Received Date** | **: 3 September 2026** |
| **Submission Due Date** | **: 23 October 2026** |
| **Lecturer** | **: Dr. Jamal Abdullahi** |
| **Weightage** | **: 15%** |
| **Semester** | **: SEPTEMBER 2026** |

**Instruction to students:**

- This is a GROUP assignment.
- Complete this cover sheet and attach it to your assignment (first page).

| **Student declaration:** | | |
|---|---|---|
| ***I declare that:***<br>· ***This assignment is my/our own work.***<br>· ***I/we understand what is meant by plagiarism.***<br>· ***My lecturer has the right to deduct my marks in case of:***<br>- ***Late submission***<br>- ***Any plagiarism found in my assignment.*** | | |
| **NAME:**<br>**SARMITA DEVADAS**<br>**MATRICS NO:**<br>**012025092104** | **NAME:**<br>**MUHAMMAD HAFIZ HAIKAL BIN MOHAMMED NAZIR**<br>**MATRICS NO:**<br>**012026020251** | **NAME:**<br>**MUHAMMAD FAKHRULLAH**<br>**MATRICS NO:**<br>**012026070691** |
|  |  |  |

# Table of Contents


**1.0 Introduction** ..... 3
**2.0 Class Attributes** ..... 3
**2.1 Description of Attributes** ..... 3
**2.2 Java Implementation of Attributes** ..... 4
**2.3 Encapsulation** ..... 4
**3.0 Methods** ..... 5
**3.1 Constructors** ..... 5
**3.2 Getters and Setters** ..... 5
**3.3 Additional Methods** ..... 5
**4.0 Class and Main Class** ..... 6
**4.1 Structure of the TicketSystem Class** ..... 6
**4.2 The Main Class** ..... 6
**4.3 Control Structures** ..... 6
**4.4 Interaction with the Object** ..... 7
**5.0 Source Code** ..... 7
**6.0 Program Output** ..... 12
**7.0 Conclusion** ..... 13

# 1.0 Introduction

The Ticketing System is an application created using the Java programming language and the Object-Oriented Programming (OOP) approach. It manages ticket details such as the ticket ID, event details, customer details, ticket price and booking status.

This assignment designs a TicketSystem class based on the concept of encapsulation. The class contains private attributes, two constructors, getter and setter methods, and additional methods that perform operations on a ticket. A separate Main class is used to interact with the object. The work was divided as follows: Sarmita wrote the attributes, Hafiz wrote the methods, and Fakhrul wrote the class structure and the main class.

# 2.0 Class Attributes

The TicketSystem class has nine attributes that represent the features of a ticket. They are the ticket ID, event name, customer name, event date, seat number, ticket price, quantity, booking status and the date and time of the booking. All attributes are declared private for encapsulation, so other classes cannot read or change them directly.

## 2.1 Description of Attributes

| Attributes | Data Type | Description |
|---|---|---|
| ticketID | String | Stores the unique identification number of a ticket |
| eventName | String | Stores the name of the event |
| customerName | String | Stores the name of the customer who buys the ticket |
| eventDate | String | Stores the date of the event |
| seatNumber | int | Stores the seat number assigned to the customer |
| ticketPrice | double | Stores the price of one ticket |
| quantity | int | Stores the number of tickets purchased |
| isBooked | boolean | Indicates whether the ticket has been booked |
| dateTime | LocalDateTime | Stores the date and time of the booking, used to generate the ticket ID |

## 2.2 Java Implementation of Attributes

The attributes are declared using the private access modifier, as shown below.

```
private String ticketID;
private String eventName;
private String customerName;
private String eventDate;
private int seatNumber;
private double ticketPrice;
private int quantity;
private boolean isBooked;
private LocalDateTime dateTime;
```

Because the attributes are private, they can only be accessed through the getter and setter methods described in Section 3.0.

## 2.3 Encapsulation

Encapsulation is one of the main principles of OOP. It means keeping the data of a class hidden and allowing access only through controlled methods.

In the TicketSystem class, all nine attributes are private, so outside classes such as Main cannot change them directly. For example, ticketPrice can only be read with getTicketPrice() and changed with setTicketPrice(). Controlled access also allows checks to be made. In our program, bookTicket() rejects an empty customer name, so a ticket can never be booked without a name. Validation could also be added to a setter in the future, for example to reject a negative ticket price. This protects the ticket data and makes the program more reliable.

# 3.0 Methods

## 3.1 Constructors

The TicketSystem class has two constructors. The default constructor takes no parameters and gives every attribute a safe starting value (“Not set” for text, 0 for numbers, and false for isBooked), so an object is never left with empty values. The parameterized constructor takes six values (eventName, customerName, eventDate, seatNumber, ticketPrice and quantity) and assigns them to the attributes using this keyword. isBooked starts as false and ticketID as “N/A” because the ticket has not been booked yet.

## 3.2 Getters and Setters

Eight of the nine attributes have one getter and one setter (eight of each). The dateTime attribute is used only inside the class to generate the ticket ID, so it does not need a getter or setter. Getters return the value of an attribute, and setters change it. Because all attributes are private, other classes such as Main can only read or change them through these methods, which is the principle of encapsulation. For example, Main uses setEventName("F1") to set the event and getTicketID() to display the ticket ID.

## 3.3 Additional Methods

| Method | Returns | Purpose |
|---|---|---|
| calculateTotal() | double | Multiplies ticketPrice by quantity to get the total amount to pay. |
| bookTicket(String name) | boolean | Checks that the name is not empty. If valid, it stores the name, marks the ticket as booked, records the time, generates the ticket ID (format TCK-date-time-seat) and returns true. Otherwise, it returns false. |
| cancelBooking() | void | If the ticket is booked, it sets isBooked to false and resets the customer’s name and ticket ID. If not, it prints that there is no booking to cancel. |

# 4.0 Class and Main Class

## 4.1 Structure of the TicketSystem Class

The TicketSystem class represents one ticket booking. It contains the nine private attributes, two constructors, eight getters, eight setters and three additional methods (calculateTotal(), bookTicket() and cancelBooking()). The class only stores and processes ticket data. It does not read input or display menus, which are the job of the Main class. This keeps each class responsible for one thing and makes TicketSystem easy to reuse.

## 4.2 The Main Class

The Main class is the program that uses the TicketSystem object. It creates the object using the default constructor and then:

1. Asks the user to choose an event (F1 or F2).
2. Sets the event name, date and price using setters.
3. Ask for the number of tickets.
4. Books each ticket by asking for the customer's name, then shows the ticket details using getters.
5. Shows the total price and offers to cancel the last booking.
6. Demonstrates the parameterized constructor with a fixed sample ticket.

## 4.3 Control Structures

| Control structure | Where it is used | Purpose |
|---|---|---|
| do-while loop | Choosing the event | Repeats until the user enters 1 or 2 |
| switch | After choosing the event | Sets the event name, date and price |
| do-while loop | Entering quantity | Repeats until quantity is at least 1 |
| for loop | Booking tickets | Books one ticket at a time, from 1 to the quantity |
| do-while loop | Entering name | Repeats until a valid (non-empty) name is entered |
| if | After a name is rejected | Shows an error message |
| if | After booking | Prints the ticket details if isBooked() is true |
| if | Cancel question | Cancels the booking if the user enters "y" |

### 4.4 Interaction with the Object

Main interacts with the object through its public methods only. It uses setters (setEventName(), setEventDate(), setTicketPrice(), setQuantity(), setSeatNumber()), getters (getTicketID(), getCustomerName(), getSeatNumber(), getTicketPrice(), isBooked()) and the additional methods bookTicket(), calculateTotal() and cancelBooking(). It never accesses an attribute directly, which follows the principle of encapsulation.

## 5.0 Source Code

```
//TicketSystem.java

import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;

// TicketSystem class: represents one event ticket booking.
// All attributes are private (encapsulation) and accessed through
getters/setters.

public class TicketSystem {

   private String ticketID;
   private String eventName;
   private String customerName;
   private String eventDate;
   private int seatNumber;
   private double ticketPrice;
   private int quantity;
   private boolean isBooked;
   private LocalDateTime dateTime;


    // 1. Default constructor: sets starting values
    public TicketSystem() {
        this.ticketID = "N/A";
        this.eventName = "Not set";
        this.customerName = "Not set";
        this.eventDate = "Not set";
        this.seatNumber = 0;
        this.ticketPrice = 0.0;
        this.quantity = 0;
        this.isBooked = false;
        this.dateTime = LocalDateTime.now();
    }

    // 2. Parameterized constructor: sets all attributes from the values
given
    public TicketSystem(String eventName, String customerName, String
eventDate,int seatNumber, double ticketPrice, int quantity) {
        this.eventName = eventName;
        this.customerName = customerName;
        this.eventDate = eventDate;
```

```
        this.seatNumber = seatNumber;
        this.ticketPrice = ticketPrice;
        this.quantity = quantity;
        this.isBooked = false;
        this.dateTime = LocalDateTime.now();
        this.ticketID = "N/A";
    }

    // Getters: return the value of each attribute
    public String getTicketID() { return ticketID; }
    public String getEventName() { return eventName; }
    public String getCustomerName() { return customerName; }
    public String getEventDate() { return eventDate; }
    public int getSeatNumber() { return seatNumber; }
    public double getTicketPrice() { return ticketPrice; }
    public int getQuantity() { return quantity; }
    public boolean isBooked() { return isBooked; }

    // Setters: change the value of each attribute
    public void setTicketID(String ticketID) { this.ticketID = ticketID;
}
    public void setEventName(String eventName) { this.eventName =
eventName; }
    public void setCustomerName(String customerName) { this.customerName
= customerName; }
    public void setEventDate(String eventDate) { this.eventDate =
eventDate; }
    public void setSeatNumber(int seatNumber) { this.seatNumber =
seatNumber; }
    public void setTicketPrice(double ticketPrice) { this.ticketPrice =
ticketPrice; }
    public void setQuantity(int quantity) { this.quantity = quantity; }
    public void setBooked(boolean booked) { this.isBooked = booked; }

    // Additional method 1: calculate total price (uses ticketPrice and
quantity)
    public double calculateTotal() {
        return ticketPrice * quantity;
    }

    // Additional method 2: book a ticket for a customer
    // Returns true if the name is valid, otherwise false
    public boolean bookTicket(String name) {
        if (name == null || name.trim().isEmpty()) {
            return false;                       // invalid name, not
booked
        }
        this.customerName = name.trim();
        this.isBooked = true;
        this.dateTime = LocalDateTime.now();
        DateTimeFormatter formatter =
DateTimeFormatter.ofPattern("yyyyMMdd-HHmmss");
        this.ticketID = "TCK-" + dateTime.format(formatter) + "-" +
seatNumber;
        return true;
    }
```

```
    // Additional method 3: cancel a booking (status check, then reset)
    public void cancelBooking() {
        if (isBooked) {
            isBooked = false;
            customerName = "Not set";
            ticketID = "N/A";
            System.out.println("Booking cancelled.");
        } else {
            System.out.println("No booking to cancel.");
        }
    }

}
```

```
//Main.java

import java.util.Scanner;

// Main class: lets the user book tickets using the TicketSystem object
public class Main {

    public static void main(String[] args) {
        try (Scanner scanner = new Scanner(System.in)) {
            TicketSystem ticket = new TicketSystem();       // default
constructor

            System.out.println("Ticketing System");
            System.out.println("================");
            System.out.println("Choose event:\n1. F1\n2. F2");

            // Looping + selection: repeat until the user enters 1 or 2
            int choice;
            do {
                System.out.print("Enter number event: ");
                choice = scanner.nextInt();
                if (choice != 1 && choice != 2) {
                    System.out.println("Try again...");
                }
            } while (choice != 1 && choice != 2);

            // Selection (switch): set event details using setters
               switch (choice) {
                case 1:
                    ticket.setEventName("F1");
                    ticket.setEventDate("10/10/2026");
                    ticket.setTicketPrice(2100);
                    break;
                case 2:
                    ticket.setEventName("F2");
                    ticket.setEventDate("11/11/2026");
                    ticket.setTicketPrice(1200);
                    break;
            }
```

```
            // Looping: repeat until the quantity is at least 1
            int qty;
            do {
                System.out.print("How many tickets? ");
                qty = scanner.nextInt();
                if (qty < 1) {
                    System.out.println("Quantity must be at least 1.");
                }
            } while (qty < 1);
            ticket.setQuantity(qty);
            scanner.nextLine();                              // clear the
Enter key

            System.out.println("\nEvent: " + ticket.getEventName()
                    + " on " + ticket.getEventDate());

            // Looping (for): book one ticket at a time
            for (int i = 1; i <= ticket.getQuantity(); i++) {

                ticket.setSeatNumber(i);                     // seat
number for this ticket

                boolean success;
                // Loop: keep asking until a valid (non-empty) name is
entered
                do {
                    System.out.print("\nTicket " + i + " of " +
ticket.getQuantity()
                            + " - enter name: ");
                    String name = scanner.nextLine();
                    success = ticket.bookTicket(name);     // additional
method 2
                    // If the name was rejected, tell the user to try
again
                    if (!success) {
                        System.out.println("Name cannot be empty. Try
again...");
                    }
                } while (!success);

                // Selection (if): check the booking status
                if (ticket.isBooked()) {
                    System.out.println("Ticket ID : " +
ticket.getTicketID());
                    System.out.println("Name      : " +
ticket.getCustomerName());
                    System.out.println("Seat      : " +
ticket.getSeatNumber());
                    System.out.println("Price     : RM" +
ticket.getTicketPrice());
                }
            }

            // Additional method 1: total price
            System.out.println("\nTotal you pay is RM" +
ticket.calculateTotal());
```

```
            // Additional method 3: cancel the last booking if the user
wants
            System.out.print("Cancel the last booking? (y/n): ");
            String answer = scanner.nextLine();
            // If the user typed y, cancel the booking
            if (answer.equalsIgnoreCase("y")) {
                ticket.cancelBooking();
            }

            // Demo of the parameterized constructor (fixed example
data, not user input)
            System.out.println("\n--- Parameterized constructor demo ---
");
            TicketSystem sample = new TicketSystem("F1", "Ali",
"10/10/2026", 1, 2100, 2);
            System.out.println("Event    : " + sample.getEventName() + "
on " + sample.getEventDate());
            System.out.println("Customer : " +
sample.getCustomerName());
            System.out.println("Seat     : " + sample.getSeatNumber());
            System.out.println("Quantity : " + sample.getQuantity());
            System.out.println("Total    : RM" +
sample.calculateTotal());

            System.out.println("Thank you..");
        }
    }
}
```

# 6.0 Program Output

The output below shows booking three tickets for event F1 and the parameterized constructor demo.

```
Ticketing System
================
Choose event:
1. F1
2. F2
Enter number event: 1
How many tickets? 3

Event: F1 on 10/10/2026

Ticket 1 of 3 - enter name: Hafiz
Ticket ID : TCK-20261022-231551-1
Name      : Hafiz
Seat      : 1
Price     : RM2100.0

Ticket 2 of 3 - enter name: Sarmita
Ticket ID : TCK-20261022-231600-2
Name      : Sarmita
Seat      : 2
Price     : RM2100.0

Ticket 3 of 3 - enter name: Fakhrul
Ticket ID : TCK-20261022-231606-3
Name      : Fakhrul
Seat      : 3
Price     : RM2100.0

Total you pay is RM6300.0
Cancel the last booking? (y/n): n

--- Parameterized constructor demo ---
Event    : F1 on 10/10/2026
Customer : Ali
Seat     : 1
Quantity : 2
Total    : RM4200.0
Thank you..
```

# 7.0 Conclusion

In conclusion, this assignment applied the key principles of Object-Oriented Programming (OOP) through our group's Ticketing System. By developing the TicketSystem class with private attributes, two constructors, and getters and setters, we learned how encapsulation protects data and restricts direct access to class fields. The additional methods showed how an object can perform useful operations based on its own attributes.

The Main class used selection structures (if, switch) and looping structures (for, do-while) to interact with the object in an organised way. Overall, the assignment gave us practical experience in turning object-oriented design concepts into clear, well-structured and maintainable code.